



POZNAN UNIVERSITY OF TECHNOLOGY

FACULTY OF COMPUTING AND TELECOMMUNICATION
Institute of Computing Science

Master's thesis

COMPARATIVE STUDY OF COMPATIBLE SUBSTITUTIONS OPTIMIZATION AND LATENT SPACE OPTIMIZATION IN EVOLUTIONARY DESIGN

Filip Rosiak, 151799

Supervisor
dr hab. inż. Maciej Komosiński

POZNAŃ 2026

Tutaj będzie karta pracy dyplomowej;
oryginał wstawiamy do wersji dla archiwum PP, w pozostałych kopiach wstawiamy ksero.

Abstract

This thesis investigates optimization methods for variable-length, non-positional genotypes in evolutionary design. Framsticks F1 genotypes are used as the main case study because their tree-like structure makes local genotype modifications strongly context-dependent. In such representations, the effect of a fragment depends on its surrounding structure, which makes standard mutation and crossover operators prone to disrupting useful substructures.

The work compares two complementary approaches to this problem. The first one, Compatible Substitutions Optimization (CoSO), operates directly in the discrete genotype representation and attempts to guide substitutions by estimating their contextual compatibility. The second one, Latent Space Optimization (LSO), maps genotypes into a continuous latent representation learned by autoencoders and performs search in that space. The autoencoder-based part includes grammar-aware models such as TreeVAE and TransformerVAE, as well as variants using grammatical context, depth information, and fitness conditioning. The optimized candidates are decoded back into F1 genotypes and evaluated with the Framsticks fitness criterion `vertpos`.

The experimental evaluation is designed to compare CoSO, latent-space optimization methods, and standard Framsticks evolutionary operators under a common protocol. The comparison considers the quality of the best solutions found, improvement over the initial genotypes, stability across starting points, and the influence of the initial fitness range. The results are intended to show when explicit context-aware substitutions and learned continuous representations are useful, what their limitations are, and how the choice of representation affects the optimization process.

Streszczenie

Niniejsza praca dotyczy metod optymalizacji genotypów o zmiennej długości i niepozycyjnym charakterze reprezentacji w projektowaniu ewolucyjnym. Jako główny przypadek badawczy wykorzystano genotypy Framsticks F1, których drzewiasta struktura sprawia, że lokalne modyfikacje genotypu są silnie zależne od kontekstu. W takich reprezentacjach znaczenie fragmentu zależy od otaczającej go struktury, przez co standardowe operatory mutacji i krzyżowania mogą niszczyć użyteczne podstruktury.

W pracy porównywane są dwa uzupełniające się podejścia do tego problemu. Pierwsze z nich, Compatible Substitutions Optimization (CoSO), działa bezpośrednio w dyskretniej reprezentacji genotypu i wykorzystuje ocenę zgodności kontekstowej proponowanych podmian. Drugie, Latent Space Optimization (LSO), odwzorowuje genotypy do ciągłej przestrzeni ukrytej wyuczonej przez autoenkodery i prowadzi optymalizację w tej przestrzeni. Część autoenkodera obejmuje modele świadome gramatyki, takie jak TreeVAE i TransformerVAE, a także warianty wykorzystujące kontekst gramatyczny, informację o głębokości oraz warunkowanie wartością fitness. Kandydaci wygenerowani w przestrzeni ukrytej są dekodowani z powrotem do genotypów F1 i oceniani przy użyciu kryterium *vertpos* w środowisku Framsticks.

Ewaluacja eksperymentalna została zaprojektowana tak, aby porównać CoSO, metody optymalizacji w przestrzeni ukrytej oraz standardowe operatory ewolucyjne Framsticks we wspólnym protokole. Porównanie obejmuje jakość najlepszych znalezionych rozwiązań, poprawę względem genotypów startowych, stabilność między różnymi punktami startowymi oraz wpływ początkowego zakresu fitness. Celem analizy jest określenie, kiedy jawne podmiany zgodne z kontekstem oraz wyuczone reprezentacje ciągłe są użyteczne, jakie są ich ograniczenia oraz w jaki sposób wybór reprezentacji wpływa na proces optymalizacji.

Contents

Abstract	I
Streszczenie	II
1 Introduction	1
1.1 Problem statement	1
1.2 Motivation	2
1.3 Aim and scope of the thesis	3
1.4 Proposed research approach	3
1.5 Thesis structure	4
2 Literature and Background	5
2.1 Evolutionary design and variable-length genotypes	5
2.2 Framsticks and the F1 genetic representation	6
2.3 Context-aware genotype modification	7
2.4 Compatible Substitutions Optimization	7
2.5 Autoencoders and variational autoencoders	7
2.6 Latent Space Optimization	8
2.7 Summary	9
3 Methods and Implemented Components	10
3.1 Dataset and fitness evaluation	10
3.2 Grammar-based representation of F1 genotypes	11
3.3 Compatible Substitutions Optimization module	12
3.4 Autoencoder-based genotype representation	12
3.5 Latent-space optimization methods	14
3.6 Baseline evolutionary methods	15
3.7 Implementation architecture	15
4 Experimental Design	16
4.1 Research questions	16
4.2 Compared methods	16
4.3 Benchmark setup	16
4.4 Evaluation metrics	17
4.5 Experimental protocol	19
4.6 Reproducibility and computational setup	19
5 Experimental Results	20
5.1 Autoencoder reconstruction and validity results	20

5.2	CoSO optimization results	20
5.3	Latent-space optimization results	20
5.4	Comparison with the Framsticks evolutionary baseline	20
5.5	Comparison between CoSO and Latent Space Optimization	20
5.6	Analysis by initial fitness ranges	20
5.7	Summary of results	20
6	Discussion and Conclusions	21
6.1	Summary of experimental results	21
6.2	Strengths and limitations of Compatible Substitutions Optimization	21
6.3	Strengths and limitations of Latent Space Optimization	21
6.4	Comparative discussion	21
6.5	Threats to validity	21
6.6	Conclusions	21
6.7	Future work	21
	Bibliography	22
A	Supplementary Materials	23
A.1	Additional experimental results	23
A.2	Configuration details	23
A.3	Examples of optimized genotypes	23

Chapter 1

Introduction

1.1 Problem statement

Evolutionary design studies the automatic generation and improvement of artefacts whose structure is evaluated by a task-dependent fitness function. In this thesis, the considered artefacts are virtual three-dimensional organisms represented as Framsticks F1 genotypes and evaluated with respect to the **vertpos** criterion. The genotype is not a fixed-length vector in which every coordinate has an independent and stable meaning. Instead, an F1 genotype is a variable-length symbolic expression with a branching, tree-like interpretation. The F1 representation combines sequential elements, branches, cores, and modifiers, which together define the syntactic structure from which a phenotype is constructed. Consequently, the same local symbol or fragment may have a different effect depending on its position and surrounding genotype context.

This representation creates a difficult optimization problem. Search is performed in a discrete and highly constrained space, because candidate strings must remain syntactically valid and must decode into valid Framsticks structures accepted for evaluation before their fitness can be assessed. Validity and semantics are therefore not properties of isolated characters only, but of complete structured expressions. A local modification that is syntactically acceptable in one part of a genotype may be invalid, neutral, or harmful in another part. This context dependence limits the effectiveness of naive edits and makes the definition of useful neighbourhoods non-trivial.

The standard evolutionary approach in Framsticks provides a natural and strong baseline for this type of problem. However, the thesis investigates whether additional representations or modification principles can improve the search process or at least offer competitive alternatives. The first approach, Compatible Substitutions Optimization (CoSO), addresses the problem directly in the discrete genotype representation by considering whether a proposed substitution is compatible with the context in which it is inserted. In this thesis, CoSO is treated as a context-aware family of genotype-editing methods.

The second approach, Latent Space Optimization (LSO), attempts to transform the same structured genotype search problem into optimization over a fixed-dimensional continuous representation learned by autoencoders. In this setting, a model encodes an F1 genotype into a latent vector, search operators modify vectors in the latent space, and the decoder maps the resulting vectors back to candidate genotypes that can be evaluated in Framsticks. This strategy is promising because it may introduce smoother neighbourhood relations than raw symbolic edits, but it also introduces new risks: decoded candidates may be invalid, the learned geometry may not correspond to fitness-improving changes, and good reconstruction quality does not by itself guarantee good optimization performance.

The problem addressed in this thesis is therefore the comparative assessment of two ways of handling context-dependent, variable-length genotypes in evolutionary design. CoSO represents an explicit, discrete, context-aware strategy, whereas LSO represents an implicit, learned, continuous strategy. The central question is not whether either approach universally replaces the standard Framsticks evolutionary algorithm, but whether and when these approaches can provide competitive search behaviour relative to this baseline for F1 genotypes optimized for `vertpos`. Detailed evaluation criteria and experimental comparisons are defined in the later chapters.

1.2 Motivation

Evolutionary design is motivated by the possibility of producing complex structures automatically, without requiring a designer to specify every morphological detail in advance. Systems such as Framsticks are particularly suitable for this type of research because they combine an explicit genotype representation, a simulation-based phenotype evaluation, and open-ended structural variation. At the same time, these properties make optimization difficult. A useful method should not only discover high-fitness organisms, but also preserve enough syntactic and structural coherence for the search process to remain productive over many generations.

The F1 representation highlights a broader challenge that appears in many generative and artificial-life settings: candidate solutions are neither simple vectors nor independent collections of parameters. They are structured objects whose local fragments interact through grammar, branching, and phenotype construction. This makes the design of variation operators especially important. Random local changes may explore the space widely, but they can also destroy useful substructures or produce candidates that cannot be decoded into valid structures for simulation-based evaluation. Therefore, there is practical motivation to study methods that use more information about the representation than purely position-independent mutation.

Compatible Substitutions Optimization (CoSO) is motivated by this need for explicit context awareness. Instead of treating a substitution as an isolated edit, CoSO considers whether the inserted fragment is compatible with the surrounding genotype context. This perspective is methodologically useful because it remains close to the original discrete representation and can be interpreted in terms of concrete genotype fragments. In evolutionary design, such interpretability is valuable: it can support explanations of why certain modifications may be more or less suitable in a given context. However, the usefulness of this idea must be assessed empirically, because compatibility with the local context does not automatically imply improvement in a simulation-based fitness criterion such as `vertpos`.

Latent Space Optimization (LSO) is motivated by a complementary observation. Instead of defining all useful neighbourhood relations manually in the symbolic genotype space, an autoencoder may learn a continuous representation in which related genotypes are mapped to nearby vectors. If this representation captures relevant regularities of F1 genotypes, standard continuous or population-based search operators can be applied in the latent space and then decoded back to candidate genotypes. Models that respect syntactic and structural regularities are especially relevant in this setting, but the methodological details belong to the later background and methods chapters. Nevertheless, LSO also requires careful evaluation: reconstruction quality, syntactic validity, and optimization performance are related but distinct properties.

The motivation for comparing CoSO and LSO is therefore not that one approach is expected to universally replace the other or to dominate the standard Framsticks evolutionary algorithm. Rather, they represent two different ways of addressing the same structural difficulty. CoSO makes context dependence explicit in the discrete search space, whereas LSO attempts to learn a

continuous search space in which some of this context dependence may be embedded implicitly. Studying both approaches under a common case study can clarify which benefits come from explicit compatibility information, which come from learned representations, and which limitations remain inherent to the optimization of variable-length F1 genotypes.

1.3 Aim and scope of the thesis

The main aim of this thesis is to compare two representation-aware approaches to optimization in evolutionary design: Compatible Substitutions Optimization (CoSO) and Latent Space Optimization (LSO). Both approaches address the same underlying difficulty of the Framsticks F1 representation, namely that useful modifications of a genotype depend on syntax, branching structure, and surrounding context. The comparison is not intended to prove that either approach universally replaces the standard Framsticks evolutionary algorithm. Instead, the thesis aims to determine how explicit context-aware substitutions and learned continuous representations behave when they are applied to the same class of variable-length genotypes and evaluated with the same objective.

The empirical scope of the work is limited to Framsticks F1 genotypes optimized for the **vertpos** fitness criterion. This restriction makes it possible to define a common case study in which all candidate solutions are ultimately evaluated by the same simulation-based criterion. Within this scope, CoSO represents the discrete and explicit direction of the study, while LSO represents the learned continuous-representation direction.

The LSO part of the thesis covers preparing F1 genotype data for autoencoder training, representing genotypes with character-based or grammar-aware encodings, training selected autoencoder architectures, decoding latent vectors back into F1 genotypes, and applying optimization methods in the learned latent space. The autoencoder-based investigation focuses on whether the learned representation can support productive search, not only on whether it reconstructs training examples accurately.

The thesis also includes a baseline comparison with Framsticks EA. This baseline is important because it represents the native evolutionary search procedure for the considered environment and provides a practical reference point for both CoSO and LSO. The intended outcome of the study is therefore a comparative assessment of solution quality, validity, stability, and dependence on starting conditions, rather than a single claim that one method is always superior.

1.4 Proposed research approach

The research approach follows a common experimental case study centered on F1 genotypes and the **vertpos** objective. The first step is to establish the representation and evaluation setting: genotypes are treated as structured symbolic expressions, and candidate solutions are evaluated through Framsticks. This provides a shared basis for comparing methods that operate in different search spaces.

For CoSO, the thesis considers optimization through context-aware substitutions in the original discrete genotype representation. This approach is analysed as an explicit attempt to use information about the compatibility of genotype fragments with their surrounding context.

For LSO, the research approach consists of learning continuous representations of F1 genotypes with autoencoders and then applying search operators to vectors in the latent space. A candidate latent vector is decoded into an F1 genotype and evaluated with the same Framsticks fitness criterion as the baseline. This makes it possible to compare latent-space search with di-

rect genotype-level evolution while preserving a common final evaluation procedure. The implemented LSO component considers several families of autoencoder models and several continuous or population-based optimization strategies, but their technical details are introduced only in the methods and experimental chapters.

The comparison is designed to separate three related questions. First, it examines whether generated or modified genotypes remain valid and evaluable. Second, it assesses whether the search process improves the fitness of starting genotypes. Third, it compares the stability of methods across different starting conditions. This structure reflects the fact that a representation can be syntactically convenient without necessarily producing better optimization results, and that a high-performing single run is not sufficient evidence of a stable optimization method.

1.5 Thesis structure

The remainder of the thesis is organized as follows. Chapter 2 presents the literature and background required for the study. It introduces evolutionary design with variable-length genotypes, the Framsticks environment and the F1 representation, context-aware genotype modification, Compatible Substitutions Optimization, autoencoders, variational autoencoders, and the general idea of Latent Space Optimization.

Chapter 3 describes the methods and implemented components. It covers the dataset and fitness evaluation procedure, the grammar-based handling of F1 genotypes, the CoSO component, the autoencoder representations, latent-space optimization methods, baseline evolutionary methods, and the software architecture used to run the experiments.

Chapter 4 defines the experimental design. It formulates the research questions, identifies the compared methods, describes the benchmark setup and evaluation metrics, and explains the experimental protocol and reproducibility considerations. Chapter 5 reports the experimental results, including reconstruction and validity analyses for autoencoders, CoSO results, latent-space optimization results, comparisons with Framsticks EA, a direct comparison between CoSO and Latent Space Optimization, and analyses conditioned on the initial fitness range.

Chapter 6 discusses the results and concludes the thesis. It summarizes the main findings, analyses the strengths and limitations of CoSO and LSO, compares the two approaches, discusses threats to validity, and outlines possible directions for future work. Supplementary material is provided in the appendix where appropriate.

Chapter 2

Literature and Background

2.1 Evolutionary design and variable-length genotypes

Evolutionary design is the use of computational search to automatically generate or improve artefacts whose quality can be assessed by an objective function. In contrast to conventional optimization problems with a small number of predefined numerical parameters, evolutionary design often concerns structured objects: shapes, mechanisms, controllers, artificial organisms, or other constructions whose form is not known in advance. The search process therefore has to explore both parameter values and structural alternatives. Candidate solutions are usually represented as genotypes, transformed into phenotypes, and evaluated according to a task-dependent fitness criterion. Selection and variation operators then bias the population toward solutions with better observed performance.

A distinctive feature of many evolutionary design problems is that the genotype does not have a fixed and directly positional interpretation. In a fixed-length vector representation, the meaning of a coordinate is typically stable across all individuals: the same index denotes the same parameter. By contrast, a variable-length genotype can contain fragments whose role depends on their position, nesting, and surrounding symbols. Such genotypes may encode branching morphology, repeated substructures, hierarchical components, or program-like construction rules. Their length may change during mutation and recombination, and a local edit can alter not only one parameter but also the interpretation of a larger downstream structure.

This property makes evolutionary design more expressive, but it also makes optimization more difficult. The search space becomes discrete, irregular, and strongly constrained by syntactic and semantic validity. Some candidate strings may not describe any admissible phenotype, while others may be valid but differ greatly in behaviour after only a small textual change. Furthermore, the contribution of a genotype fragment is often context-dependent: a fragment that is useful in one structural location can be neutral, invalid, or harmful in another. This context dependence is closely related to epistasis, understood broadly as interaction between parts of the representation, where the effect of modifying one part depends on other parts of the genotype.

Simulation-based optimization adds another layer of complexity. In this setting, fitness is not obtained from a closed-form mathematical expression, but from a simulation that constructs and evaluates the phenotype. This is typical for artificial-life and evolutionary robotics environments, where the performance of a morphology may depend on physical interaction, dynamics, control, or environmental conditions. Simulation enables the assessment of complex behaviours, but it also makes each evaluation relatively expensive and sometimes noisy. As a result, variation operators should ideally produce candidates that are not only diverse, but also valid and promising enough

to justify evaluation.

The representation used in this thesis belongs to this class of problems. Framsticks F1 genotypes are variable-length symbolic descriptions of virtual organisms, and their fitness is obtained after phenotype construction and simulation. The **vertpos** criterion considered later in the thesis is therefore an evaluated property of the resulting organism, not a simple property of the raw character string. This distinction is important because a method that performs well at modifying genotype text does not necessarily perform well at improving simulated fitness. Conversely, an optimization method may be valuable if it discovers modifications that preserve validity and produce useful phenotypic effects, even when the mapping from genotype to fitness is highly indirect.

The background developed in this section motivates the two representation-aware directions studied in the thesis. Compatible Substitutions Optimization (CoSO) addresses the difficulty directly in the discrete genotype representation by considering compatibility between a substitution and its context. Latent Space Optimization (LSO) approaches the same difficulty by learning a continuous representation in which variable-length genotypes are mapped to fixed-dimensional vectors before search is performed. Both directions can be understood as attempts to define more useful neighbourhoods for structured genotypes than those obtained from naive position-independent edits. The following sections introduce the concrete Framsticks F1 representation and then discuss these two approaches in greater detail.

2.2 Framsticks and the F1 genetic representation

Framsticks is an artificial-life and evolutionary-design environment in which virtual organisms are described by genotypes, developed into phenotypes, simulated, and evaluated according to selected criteria. For the purposes of this thesis, Framsticks provides both the design domain and the final evaluator of candidate solutions. The methods compared later operate on, or generate, symbolic genotype descriptions, but their usefulness is ultimately determined by whether these descriptions can be interpreted as valid organisms and assessed in simulation. The environment therefore connects representation-level search with phenotype-level evaluation.

The representation considered in this thesis is Framsticks F1. At a conceptual level, an F1 genotype is a textual description of an organism rather than a fixed list of numerical parameters. Its symbols specify structural components and modifiers, and the complete string is interpreted through a genotype-to-phenotype mapping. This mapping is indirect: the fitness of a candidate is not defined by the string alone, but by the organism produced from it and by the behaviour of this organism during simulation. The later use of the **vertpos** criterion should therefore be understood as evaluation of a constructed phenotype, not as a direct score assigned to individual characters.

F1 genotypes are variable-length expressions. This property is important because evolutionary variation can change not only parameter values, but also the amount and organization of encoded structure. Longer or shorter genotypes may describe organisms with different numbers of components, different branching patterns, or different modifier sequences. Consequently, the search space is not a simple Euclidean space with a fixed dimensionality. It is a discrete structured space in which insertion, deletion, and substitution can change the interpretation of neighbouring fragments and, in some cases, larger parts of the organism.

The F1 representation also contains branching and nested descriptions. In simplified grammatical terms, genotype strings may be understood as sequences of elements, where elements can contain a core component, optional modifiers, and branch expressions enclosed in parentheses. Branches make the representation expressive, because they allow the genotype to describe articulated or tree-like morphology. At the same time, they impose syntactic constraints: parentheses,

separators, branch contents, and modifier positions must be arranged in forms that can be parsed and developed into a phenotype. A candidate that violates these constraints may be useless for simulation-based optimization because it cannot be evaluated in the intended way.

Syntactic validity is therefore a central issue for both approaches studied in the thesis. In a direct genotype-level method, a local edit must preserve enough structure for the resulting expression to remain acceptable. In an autoencoder-based method, a decoder must produce strings that are not merely character sequences, but valid F1 genotypes. Validity is not equivalent to good performance, but invalidity prevents meaningful phenotype evaluation. This distinction motivates the later separation between reconstruction or validity analysis and actual optimization performance.

Another important property of F1 is context sensitivity. The effect of a symbol or fragment depends on where it appears in the genotype and on the structural role of the surrounding expression. A modifier before a core component, a symbol inside a branch, and a separator between branches are not interchangeable even if they are all local textual tokens. Therefore, edits that ignore context may easily produce invalid genotypes or changes whose phenotypic consequences are difficult to control. This observation prepares the ground for context-aware genotype modification and for Compatible Substitutions Optimization, where the compatibility of a substitution with its surrounding expression is treated as an essential concern.

The same representational properties also motivate Latent Space Optimization. Since F1 genotypes are variable-length, branching, and syntactically constrained, mapping them to fixed-dimensional continuous vectors is non-trivial. A useful latent representation should preserve enough information about the genotype structure to allow decoding back into valid candidates, while also arranging genotypes in a way that supports search. The background role of the F1 representation is thus twofold: it defines the concrete optimization domain of the thesis, and it explains why both explicit context-aware edits and learned continuous representations are relevant alternatives to naive character-level modification.

2.3 Context-aware genotype modification

2.4 Compatible Substitutions Optimization

2.5 Autoencoders and variational autoencoders

Autoencoders are representation-learning models trained to reproduce their inputs after passing them through an intermediate code. The model is usually divided into an encoder and a decoder. The encoder maps an input object to a latent representation, while the decoder attempts to reconstruct the original object from this representation. If the latent representation is constrained to be lower-dimensional, regularized, or otherwise information-limited, the model is encouraged to capture regularities that are useful for reconstruction rather than simply copying the input [11, 4].

This encoder–decoder structure is particularly relevant for the study of structured genotypes. A genotype such as a Framsticks F1 string is variable-length and symbolic, whereas many optimization methods operate most naturally on fixed-size numerical vectors. An autoencoder can therefore be used as a bridge between these two forms. The encoder converts a genotype into a vector in the latent space, and the decoder converts a latent vector back into a candidate genotype. In this thesis, this idea is important not because reconstruction is an end in itself, but because reconstruction defines whether the learned representation can be used as a search space for genotype generation and modification.

Variational autoencoders extend this idea by making the latent representation probabilistic. Instead of encoding an input as a single unconstrained vector only, a variational autoencoder represents the input by parameters of a latent distribution, typically mean and variance parameters, and samples a latent vector from this distribution during training. The training objective combines reconstruction accuracy with a regularization term that encourages the latent distribution to remain close to a simple prior, often a standard normal distribution. This regularization is intended to make the latent space more continuous and more suitable for sampling, interpolation, and optimization than an arbitrary collection of isolated codes [6, 9].

For continuous data, reconstruction can often be measured by numerical distance. For discrete structured objects, however, reconstruction quality has additional dimensions. A decoded sequence may differ from the input by only a small number of symbols and still be syntactically invalid. Conversely, a reconstruction that is textually different from the input may still describe a valid and potentially useful organism. Therefore, when autoencoders are applied to F1 genotypes, the central concerns are not limited to token-level accuracy. The decoded output must also respect the constraints of the representation so that it can be interpreted as a valid Framsticks genotype and evaluated with a criterion such as `vertpos`.

This validity issue explains why grammar- and structure-aware decoders are relevant. A purely character-level decoder treats generation mainly as a sequence prediction task and must learn syntactic constraints indirectly from examples. This may be sufficient for simple patterns, but it provides no explicit guarantee that parentheses, branches, modifiers, and core elements will appear in admissible combinations. By contrast, grammar-aware representations can decode sequences of production rules rather than raw characters, and masked variants can restrict choices to rules that are valid for the currently expanded nonterminal [7]. Tree-oriented variants can additionally reflect the fact that F1 genotypes contain branching structure rather than only a flat string. Such mechanisms do not by themselves guarantee high fitness, but they can reduce the burden placed on the model by embedding part of the known syntax into the decoder.

In the context of this thesis, autoencoders and variational autoencoders therefore provide the background for the Latent Space Optimization part of the comparison. They define a learned continuous representation of F1 genotypes that is different from the explicit discrete representation used by context-aware substitutions and CoSO. The usefulness of this representation depends on several separate properties: whether inputs can be encoded and reconstructed, whether decoded samples remain valid genotypes, whether nearby latent points correspond to related genotype structures, and whether movement in the latent space is correlated with changes in simulation-based fitness. These properties are connected, but none of them alone is sufficient to establish that an autoencoder is useful for optimization.

2.6 Latent Space Optimization

Latent Space Optimization (LSO) is the use of a learned latent representation as the domain in which search is performed. Instead of applying variation directly to the original object, an object is encoded into a continuous vector, optimization modifies or samples vectors in this latent space, and each candidate vector is decoded back into the original representation before evaluation. Related work has used learned continuous representations for molecular design, program synthesis, and evolutionary search in learned representations, illustrating the general idea that an autoencoder or another generative model can define a search space for structured objects [2, 5, 8, 1]. In the setting considered in this thesis, the original objects are Framsticks F1 genotypes, and the final objective

remains the true simulation-based **vertpos** fitness. The latent vector is therefore an intermediate representation for proposing candidates, not a replacement for evaluation in Framsticks.

The motivation for LSO is that a learned continuous representation may provide a more compact or smoother search domain than the raw genotype space. F1 genotypes are variable-length symbolic expressions with branching syntax and context-dependent effects. Direct edits in such a space can easily produce invalid candidates or large unpredictable phenotypic changes. If an autoencoder has learned useful regularities of the genotype distribution, nearby latent vectors may decode to related genotypes, and continuous perturbations may correspond to moderate structural changes. This can make it possible to apply optimization techniques that are difficult to define directly on raw F1 strings.

The general LSO loop consists of four conceptual steps. First, one or more starting genotypes are encoded into latent vectors. Second, an optimizer proposes new latent vectors by mutation, recombination, distribution update, or another continuous-space operation. Third, each proposed vector is decoded into an F1 genotype. Fourth, the decoded genotype is checked for validity and evaluated in Framsticks, where its **vertpos** value provides feedback to the optimizer. This loop preserves a common final evaluation criterion with direct evolutionary search: regardless of how a candidate is generated, it is useful only if it decodes into a genotype that can be evaluated and achieves good simulated performance.

Several optimizer families can be used within this framework. An evolutionary search in latent space keeps a population of latent vectors, selects better candidates according to decoded fitness, and creates new vectors through latent mutation, interpolation-like crossover, or related variation operators. The Cross-Entropy Method (CEM) instead maintains a sampling distribution over the latent space, evaluates a population of samples, and updates the distribution toward the best-performing elite samples [10]. Covariance Matrix Adaptation Evolution Strategy (CMA-ES) is another population-based continuous optimizer that adapts both the mean and covariance of its search distribution [3]. These methods differ in how they use fitness feedback, but in LSO they share the same requirement: every promising latent candidate must ultimately be decoded into a valid genotype and evaluated by the original objective.

LSO may help with structured genotypes when the latent representation captures properties that are relevant to both syntax and fitness. A compact latent space can reduce the apparent dimensionality of the search, and a regularized decoder can bias generated candidates toward patterns observed in valid genotypes. This can be advantageous when direct random edits are too disruptive. Moreover, continuous search operators can explore directions, mixtures, and local neighbourhoods in a way that has no direct analogue in symbolic genotype strings.

However, these advantages are conditional rather than guaranteed. If the autoencoder reconstructs poorly, many optimized latent vectors may decode into invalid or low-quality genotypes. If the latent space is syntactically valid but not fitness-aware, the optimizer may efficiently explore regions that reproduce common genotype patterns without improving **vertpos**. If the decoder maps large regions of latent space to similar genotypes, search may stagnate because many candidate vectors produce duplicates or near-duplicates. Conversely, if small latent changes decode into unrelated structures, the expected smoothness of the latent space is lost. For this reason, reconstruction quality, validity, diversity, and optimization performance must be analysed separately.

2.7 Summary

Chapter 3

Methods and Implemented Components

3.1 Dataset and fitness evaluation

The common empirical object used in the F1 part of the thesis is a dataset of Framsticks F1 genotype strings paired with scalar fitness values. Each record consists of a genotype written as text and a corresponding `vertpos` value separated from it by a tab character. The dataset contains 10,893 non-empty records, and all records provide a finite numeric fitness value. This format preserves a direct link between the symbolic description of an organism and the value obtained for that organism in a previous Framsticks evaluation.

In the implemented data-loading procedure, the genotype string and the fitness label are parsed as two separate fields. The genotype remains the primary structured object: it is later encoded either as a character sequence or through grammar-derived production rules, depending on the model family. The fitness value is stored as a floating-point label associated with the same genotype. For character-level processing, internal whitespace is preserved as part of the input string. For grammar-based processing and benchmark seed preparation, whitespace inside the genotype is normalized away so that the genotype can be parsed and compared consistently. This distinction reflects the fact that the dataset is used both for learning representations of genotype strings and for selecting or analysing examples according to their evaluated fitness.

The stored `vertpos` values should not be interpreted as properties directly computed from raw genotype text. A genotype first has to be accepted by the Framsticks representation, developed into a phenotype, and evaluated in simulation. The `vertpos` criterion is therefore a phenotype-level measurement returned by Framsticks for the organism described by the genotype. This distinction is important for both parts of the thesis. CoSO and Latent Space Optimization operate on genotype representations or propose genotype-level modifications, but their optimization relevance is determined only after the resulting candidates are interpreted and evaluated as Framsticks organisms.

For autoencoder training, the dataset provides examples from which the syntax and structural regularities of F1 genotypes can be learned. The implementation supports an optional train-validation split rather than relying on a separate validation file. When this split is enabled, a fixed fraction of the parsed examples is assigned to validation using a pseudorandom split controlled by the training seed, while the remaining examples form the training subset. The validation-enabled training scripts use a conventional 0.2 validation fraction. Fitness-aware variants additionally require finite stored fitness labels, because the scalar value can be used as an auxiliary target or as a conditioning signal, but the label remains tied to the original evaluated genotype.

During optimization experiments, dataset fitness values and fresh fitness evaluations play differ-

ent roles. Stored values are suitable for selecting starting genotypes, grouping examples by initial fitness, and providing labels during representation learning or analysis. However, newly produced candidates are not assigned fitness by looking up or estimating their dataset labels. Instead, candidate genotypes generated by direct genotype-level search, CoSO-style modification, or decoding from latent vectors must be evaluated through Framsticks with the same **vertpos** criterion. If a candidate cannot be evaluated successfully, it is treated as invalid for optimization purposes. This procedure keeps the final objective common across the compared methods: regardless of how a candidate is produced, its useful fitness is the true Framsticks evaluation of the phenotype generated from its F1 genotype.

3.2 Grammar-based representation of F1 genotypes

The grammar-based part of the implementation represents a Framsticks F1 genotype as a structured expression rather than only as an unconstrained sequence of characters. The raw genotype is still a variable-length string, but for grammar-aware processing it is first normalized by removing internal whitespace and then tokenized into individual F1 symbols. The resulting token sequence is accepted only if it can be parsed by the F1 context-free grammar used in the implementation. This parsing step separates syntactically admissible training examples from strings that cannot be represented as grammar derivations within the configured maximum production-sequence length.

The grammar starts from the nonterminal **S** and expands it into a sequence of elements. Each element consists of zero or more modifiers followed by a core expression. The core may be the terminal **X**, which denotes the basic F1 structural symbol used in this simplified grammar, or a parenthesized branch expression. Branches are represented with parentheses and comma-separated branch tails, and several grammar rules allow empty expansions. Empty expansions are important because they make optional components expressible: a modifier sequence may end, a branch list may be absent, and a branch tail may terminate. The modifier nonterminal expands to the set of supported F1 modifier symbols, including both uppercase and lowercase variants such as **R/r**, **Q/q**, **C/c**, **L/l**, **W/w**, **M/m**, **I/i**, **F/f**, **A/a**, **S/s**, and **E/e**. At this methodological level, the grammar therefore captures the subset of F1 syntax needed by the grammar-aware models: linear sequences, optional modifier prefixes, nested parenthesized branches, branch separators, and terminal construction symbols.

After parsing, a genotype is encoded as the sequence of productions used in one parse tree. Each production is assigned an integer index, and the production sequence is padded to a fixed maximum length for batch processing. In the integer production-index encoding, positions beyond the end of the actual derivation use a separate padding index; this index is not one of the grammar productions. The one-hot grammar encoding uses a different convention: its columns correspond only to grammar productions, and padded positions activate the final production column, which corresponds to the grammar’s **Nothing -> None** production. These two related dataset encodings support different model interfaces. The index form is convenient for embedding-based sequence models, whereas the one-hot form is convenient for models that predict a distribution over grammar productions. In both cases, the object being learned is not a character string directly, but a derivation trace that can be expanded back into a genotype string.

Decoding follows the inverse idea. A stack of nonterminals is initialized with the start symbol. At each step, the next predicted production is applied to the current nonterminal, terminal symbols are emitted to the reconstructed genotype, and newly introduced nonterminals are pushed onto the stack in the order required for subsequent expansion. This stack-based procedure gives the production sequence an explicit structural interpretation. It also explains why the order of pro-

duction choices matters: the same production index can be meaningful only when the nonterminal at the top of the derivation stack has a compatible left-hand side.

The implementation therefore precomputes grammar masks from the set of productions. For every left-hand-side nonterminal, a binary mask identifies exactly the productions whose left-hand side matches that nonterminal. During masked decoding, the current derivation stack determines which nonterminal is being expanded, and the corresponding mask suppresses productions that would be syntactically incompatible at that position. When the stack is empty, the remaining sequence positions are treated as a finished derivation through the designated empty or padding state used by the corresponding encoding. This mechanism does not evaluate the biological or fitness quality of a genotype, but it prevents a decoder from selecting productions that violate the local grammar state.

Several later model variants use the same grammar representation with different amounts of structural context. Grammar-rule models operate on production sequences. Masked variants additionally use the current nonterminal to restrict admissible production choices during decoding. Tree-oriented variants reconstruct the left-hand-side context implied by the derivation stack and can provide this context to the encoder or decoder. The LHS-conditioned variants make this context explicit by embedding the currently expanded nonterminal, while LHS-depth variants also represent the current stack depth. These details belong to the autoencoder section at the architectural level, but at the representation level they all rely on the same principle: an F1 genotype is treated as a grammar derivation with a current expansion state, rather than as an unstructured string.

This representation is useful for Latent Space Optimization because latent search ultimately proposes candidates through a decoder. If the decoder is allowed to generate arbitrary symbols, many latent points can map to strings that do not respect the F1 syntax and therefore cannot be meaningfully evaluated. A production-based representation narrows the decoding problem to choosing grammar rules, and grammar masks further constrain these choices to the rules allowed by the current derivation context. Syntactic validity is not sufficient for high `vertpos` fitness, and it does not guarantee successful optimization, but it is a prerequisite for using decoded candidates as Framsticks genotypes in later evaluation and search procedures.

3.3 Compatible Substitutions Optimization module

3.4 Autoencoder-based genotype representation

The autoencoder-based representation used in the Latent Space Optimization part of the thesis maps a variable-length F1 genotype to a fixed-dimensional numerical vector and then reconstructs a genotype from that vector. In all variants, the encoder is responsible for transforming an input sequence into a latent representation, while the decoder is responsible for producing a sequence that can be interpreted as a genotype. This representation is not a replacement for the Framsticks genotype itself. It is an intermediate search representation: optimization may operate on the latent vector, but every candidate must eventually be decoded back to F1 text before it can be evaluated with the true `vertpos` criterion.

Two input families are used. The first family treats the genotype as a character sequence. A character-level dataset builds a vocabulary containing padding, start, and end symbols, followed by the characters observed in the training strings. Each genotype is encoded as a padded sequence of token indices. The corresponding character-level variational autoencoder uses recurrent sequence encoders and decoders: the encoder embeds the tokens and summarizes the sequence into the

parameters of a latent distribution, and the decoder generates output tokens autoregressively from a sampled latent vector. This variant leaves syntactic regularities to be learned from examples, because it does not explicitly know which character choices are legal in a given grammatical context.

The second family uses the grammar representation introduced in Section 3.2. A parsed genotype is encoded as a sequence of production-rule indices, optionally represented in one-hot form. The reconstruction target is therefore not directly a sequence of raw characters, but a derivation trace that can be expanded into an F1 string. This distinction is important for validity. If the decoder predicts a rule sequence that follows the derivation stack consistently, the resulting output is constrained by the same grammar that was used to encode the training examples. The decoder output is then converted to genotype text by applying predicted productions to the current nonterminal and emitting terminal symbols.

The implemented grammar-rule variational autoencoder follows the standard encoder–decoder pattern at the level of production indices. The encoder embeds the production sequence and maps it to mean and log-variance vectors. During training, a latent vector is sampled by the reparameterization mechanism, and the decoder is trained to reproduce the input production sequence. The objective combines a reconstruction term over sequence positions with a Kullback–Leibler regularization term, with a scheduled weight on the regularization term during training. Teacher forcing is used for recurrent decoders during training and is reduced over the training schedule. At the methods level, the relevant point is that the model learns a continuous latent code from which complete grammar-rule sequences can be generated.

Masked grammar variants add explicit syntactic constraints to decoding. The F1 grammar provides, for each left-hand-side nonterminal, the set of productions that may expand it. During masked decoding, the current derivation stack determines the active nonterminal, and logits for incompatible productions are suppressed. This mechanism does not guarantee that a decoded genotype has high fitness, and it does not evaluate semantic properties of the organism. It only enforces local grammatical compatibility during rule selection, which reduces the number of syntactically impossible production choices available to the decoder.

Tree-oriented variants use the fact that a production sequence implicitly defines a derivation tree. In these models, the current left-hand-side context is reconstructed from the stack of nonterminals generated by previous rules and is provided as an additional input to the encoder. Masked tree variants combine this tree-aware context with rule masks in the decoder. LHS-conditioned masked tree variants go further by giving the decoder an embedding of the currently expanded nonterminal, and LHS-depth variants additionally represent the current stack depth. These mechanisms all use the same grammar as Section 3.2, but they expose more of the derivation state to the neural model instead of requiring it to infer the state only from preceding rule indices.

Conditional extensions are also implemented for selected tree-oriented models. Fitness-conditioned variants append a scalar conditioning value to the latent vector before decoding. Structural conditional variants use a small vector of auxiliary structural descriptors rather than only a scalar fitness condition. These variants are methodological extensions of the same autoencoder design: the latent vector remains the main representation, while the condition supplies additional information to the decoder. The presence of a conditioning signal should not be interpreted as replacing true fitness evaluation, because decoded candidates are still evaluated through Framsticks during optimization.

A Transformer-based grammar variational autoencoder is included as an alternative to recurrent sequence modelling. It uses embeddings of production-rule indices together with positional encodings and Transformer encoder–decoder blocks. The encoder pools non-padding positions to obtain latent distribution parameters, and the decoder generates a production sequence from the

latent vector using a causal decoding structure. This variant keeps the same grammar-rule input and reconstruction target as other grammar-based models, but changes the sequence model used to capture dependencies between production choices.

Finally, a vector-quantized grammar autoencoder is implemented as a related representation-learning variant. Instead of sampling from a continuous variational distribution, it maps an encoded production sequence to a latent vector that is quantized through a learned codebook and then decoded back into production rules. This variant is relevant as an alternative autoencoding mechanism for grammar-rule sequences, although the main Latent Space Optimization procedures considered in this thesis operate on continuous latent vectors generated by variational models.

Across these model families, the training target remains reconstruction of the input representation: characters for the character-level model and grammar production sequences for grammar-aware models. Reconstruction, however, is only a prerequisite for later optimization. A model that reconstructs valid examples well may still define a latent space that is difficult to optimize, and a syntactically constrained decoder may still produce organisms with low `vertpos`. For this reason, autoencoder training and latent-space search are treated as separate methodological components: the autoencoder defines the representation, while the optimization methods described next decide how that representation is explored.

3.5 Latent-space optimization methods

Latent-space optimization uses a trained genotype autoencoder as a generator of candidate F1 genotypes. The optimizer itself manipulates fixed-dimensional latent vectors, but the objective value assigned to a vector is obtained only after decoding. In the implemented latent EA, CEM, and CMA-ES benchmark procedures, latent candidates are decoded as grammar-rule sequences, converted into genotype text, checked for syntactic admissibility, and evaluated in Framsticks. Character-level autoencoders remain part of the broader representation set described in Section 3.4, but they are not the decoding context used by these grammar-rule optimization loops. The scalar feedback used by the optimizer is the resulting true `vertpos` fitness, with invalid or unevaluable candidates treated as uncompetitive. Thus, the learned model proposes candidates, whereas Framsticks remains the source of fitness values.

In the seeded setting used by the implemented procedures, optimization starts from one or more existing genotypes. A starting genotype is encoded by the autoencoder, and the mean vector produced by the encoder is used as the initial latent point. This choice gives the search a concrete reference candidate rather than beginning only from an unconditional prior sample. For grammar-rule models, starting genotypes must be encodable by the grammar representation and must fit within the configured maximum production-sequence length. The decoded form of the starting latent vector can also be evaluated, because the autoencoder reconstruction of the seed may differ from the original genotype.

All latent-space methods share the same evaluation loop. A population or distribution produces a batch of latent vectors. Each vector is decoded to an F1 genotype, normalized for comparison, and rejected if it is empty, syntactically invalid, duplicated when uniqueness is required, or impossible to evaluate. Remaining candidates are evaluated by Framsticks with the `vertpos` criterion. Fitness values are cached for already evaluated genotypes so that the optimizer is not rewarded for repeatedly generating the same string. The best decoded genotype and its true fitness are tracked separately from the latent vector that produced it, because the final output of the method is an F1 genotype rather than the vector itself.

The latent-space evolutionary search maintains a population of individuals represented by both decoded genotype and latent vector. Initial population members are produced by perturbing the encoded seed vector with Gaussian noise of selected standard deviations and evaluating the decoded genotypes. In each generation, parents are selected according to true evaluated fitness. Offspring latent vectors are generated through several continuous-space operators: mutation by Gaussian perturbation, crossover-like interpolation between parent vectors, optional extrapolation along lines between parents, and directional moves biased by the difference between a high-fitness vector and another selected vector. After decoding and true evaluation, valid offspring are merged with elite survivors, and the population is truncated to the configured size.

Random immigrants are supported in the latent evolutionary search as an exploration mechanism. With a configurable probability, a child vector is sampled from a standard normal distribution rather than produced from selected parents. This operation introduces latent candidates that are not constrained to remain near the current population. Methodologically, it reduces the risk that the population collapses prematurely around a narrow decoded region, although the sampled vectors must still decode to valid and useful genotypes to affect the search.

The same evolutionary implementation can optionally re-encode accepted offspring. In this variant, a decoded genotype that passes validity and evaluation is encoded again, and the resulting latent mean replaces the raw child vector. Re-encoding projects the individual back through the autoencoder’s representation of the decoded genotype. This can make the stored latent population more consistent with the model’s encoder–decoder manifold, but it is an operator choice rather than a separate objective function.

The Cross-Entropy Method (CEM) is implemented as a distribution-based optimizer over latent vectors. Starting from the encoded seed, the method maintains a diagonal normal sampling distribution. At each iteration, it samples a population of latent candidates, decodes and evaluates them, selects an elite fraction according to `vertpos`, and updates the distribution mean and standard deviation toward the elite set. Smoothing and a minimum standard deviation are used so that the distribution does not change too abruptly or collapse completely. CEM therefore differs from the latent evolutionary search in that it does not maintain explicit genotype individuals across generations; it adapts a sampling distribution from elite latent samples.

CMA-ES is implemented as another continuous distribution-search method. It samples latent candidates around a mean vector, evaluates the decoded genotypes, and updates the search mean and covariance structure according to the best-performing samples. In this setting, the optimizer minimizes the negative of the true fitness only implicitly through the scores returned after decoding and Framsticks evaluation; it does not use gradients through the decoder or through the simulator.

The latent-space methods are therefore derivative-free black-box optimizers with respect to the final objective. The autoencoder is trained before optimization and is used only to map between F1 genotypes and latent vectors. During search, no learned surrogate fitness is used to replace Framsticks evaluation, and no gradient of `vertpos` is available. This separation is essential for the comparison performed in later chapters: differences between methods can be attributed to how they explore the learned representation and how often they produce valid, diverse, high-fitness decoded genotypes, not to a change in the underlying fitness criterion.

3.6 Baseline evolutionary methods

3.7 Implementation architecture

Chapter 4

Experimental Design

4.1 Research questions

4.2 Compared methods

4.3 Benchmark setup

The common benchmark is centred on Framsticks F1 genotypes and on the **vertpos** fitness criterion introduced in Section 3.1. The dataset is used to provide existing genotypes with known starting fitness values, but it is not used as a lookup table for the fitness of newly generated candidates. During optimization, each candidate produced by a baseline evolutionary run or by decoding a latent vector is converted to an F1 genotype and evaluated through Framsticks with the same **vertpos** criterion. Candidates that cannot be represented or evaluated successfully are treated as invalid for the corresponding search procedure.

Starting genotypes for the final paired benchmark are selected from six initial-fitness buckets. The bucket ranges are $[0.30, 0.50)$, $[0.50, 0.75)$, $[0.75, 1.00)$, $[1.00, 1.20)$, $[1.20, 1.50)$, and $[1.50, \infty)$. Twenty-five starting genotypes are sampled from each bucket, giving 150 starting points in total. The same seed manifest is used for all latent-space configurations and for the Framsticks EA baseline. This design makes the comparison paired: for a given starting genotype, latent-space results can be compared with the baseline result obtained from the same initial organism, rather than with an independently sampled baseline run.

The final Latent Space Optimization benchmark combines five selected autoencoder checkpoints with three latent-space optimizers. The checkpoint labels used in the benchmark are:

- 09_tree_vae_epoch80,
- 10_transformer_vae_epoch90,
- 02_lhs_latent256_seed42_best,
- 06_lhs_depth_seed42_best,
- fitness_aware_lhs_seed42.

For each checkpoint, three latent methods are evaluated: the latent-space evolutionary algorithm, reported as **latent_ea**; the Cross-Entropy Method, reported as **cem**; and CMA-ES, reported as **cmaes**. The resulting latent part of the benchmark therefore consists of all combinations of five checkpoints, three methods, six buckets, and twenty-five seeds per bucket.

The baseline part of the benchmark uses the standard Framsticks EA from the same starting genotypes. There is one baseline run for each of the 150 starting genotypes. The nominal search budget is kept consistent at the design level: evolutionary runs use 300 generations with population size 100, while distribution-based latent methods use 300 iterations with the same population size. Actual numbers of unique evaluated genotypes may differ between runs because invalid candidates, duplicates, and cached evaluations are handled by the optimization procedure. The number of unique evaluated genotypes is recorded for all runs, while invalid and duplicate counts are interpreted only for methods and jobs that record them; these measurements are used for later metric analysis rather than being treated as separate benchmark conditions.

The computational execution separates neural latent-space jobs and baseline evolutionary jobs only for practical reasons: latent decoding uses trained autoencoder checkpoints and is configured for accelerator-based execution, whereas the Framsticks EA baseline does not require neural inference. This distinction does not change the objective function, the starting genotypes, or the use of true Framsticks `vertpos` evaluation. It is therefore a reproducibility detail rather than an experimental factor.

4.4 Evaluation metrics

The evaluation uses separate but connected metric groups for representation learning, optimization quality, and search behaviour. This separation is important because a genotype autoencoder can reconstruct examples well without necessarily producing a latent space that is easy to optimize. Conversely, an optimization method can occasionally discover a high-fitness genotype even if its generator has limited average reconstruction accuracy. The reported metrics therefore distinguish model diagnostics from final objective performance.

For autoencoder training and checkpoint selection, the main reconstruction metric is validation exact reconstruction accuracy. A reconstruction is counted as exact when the decoded output string is identical to the corresponding reference genotype string after the representation-specific decoding step. The accuracy is the number of exact reconstructions divided by the number of validation examples. Because exact equality is a strict criterion for variable-length structured genotypes, average string similarity is also recorded. This metric averages a normalized string-similarity score over reconstructed examples and indicates partial preservation of genotype text. It should be interpreted only as a reconstruction diagnostic: a high similarity value does not imply high `vertpos`, and a low exact-reconstruction value does not by itself determine the outcome of latent-space search.

Selected checkpoints are additionally characterized by unconditional generation diagnostics. Random latent samples are decoded and classified according to whether the resulting strings are syntactically valid F1 genotypes. The validity rate is the fraction of generated samples that pass this syntactic check. Uniqueness is measured by counting distinct generated strings, and duplicate rate is the complementary fraction of repeated generated samples. For valid generated genotypes, novelty relative to the available dataset and, when evaluated, the count and summary statistics of generated `vertpos` values are auxiliary indicators of generator behaviour. These quantities help describe whether a checkpoint can produce diverse admissible genotypes, but they are not substitutes for the final seeded optimization benchmark. For fitness-guided variants, fitness-prediction error and correlation are considered only when such diagnostics are recorded or when a fitness-prediction head is used; in those cases, they are auxiliary training diagnostics rather than final optimization metrics.

The primary optimization-quality metric is the best true **vertpos** score obtained in a run. For method or configuration m and starting genotype s , this value is denoted by

$$F_{m,s}^{\text{best}}.$$

It corresponds to the best genotype returned by the run after candidate decoding, validity checks, method-specific duplicate or cache handling, and Framsticks evaluation. Improvement over the starting genotype is measured as

$$\Delta_{\text{seed}}(m, s) = F_{m,s}^{\text{best}} - F_s^{\text{seed}},$$

where F_s^{seed} is the stored **vertpos** value of the starting genotype. A complementary conservative value is also useful in interpretation:

$$F_{m,s}^{\text{best-or-source}} = \max(F_{m,s}^{\text{best}}, F_s^{\text{seed}}).$$

This value answers the question of how much would be gained if the original seed genotype were retained whenever a search run failed to improve on it.

Comparisons with the Framsticks EA baseline are paired by starting genotype. For each non-baseline method or configuration and seed, the paired difference is

$$\Delta_{\text{EA}}(m, s) = F_{m,s}^{\text{best}} - F_{\text{Framsticks EA},s}^{\text{best}}.$$

A positive difference is counted as a win against the baseline, a negative difference as a loss, and an exact zero as a tie. Aggregates over seeds include the mean, median, maximum, and variability of best scores, improvements over the starting genotype, and paired differences against the baseline. The win rate is the fraction of paired seeds for which the compared method obtains a strictly larger best score than the Framsticks EA run from the same starting genotype. If best-per-seed or portfolio-style summaries are shown later, they are interpreted only as upper-bound analyses over already completed configurations, not as independent optimization methods.

Search behaviour and stability are described by quantities recorded during each run. The number of unique evaluated genotypes is used as an effective measure of how many distinct genotype strings were assigned fitness values during search. This quantity is broadly available in the final benchmark outputs and helps compare how much unique search material each method produced under the same nominal budget. Invalid-candidate and duplicate-candidate counts are more method-dependent. They are interpreted only for methods and runs whose output explicitly records them; missing fields are not treated as zero counts. When available, invalid counts indicate decoded or modified candidates that could not be used as evaluated F1 genotypes, while duplicate counts indicate repeated normalized genotypes that did not add new search information.

The benchmark also records best-so-far histories over generations or iterations. These histories support trajectory-based analyses, such as median best **vertpos** as a function of requested evaluations, median improvement over the starting genotype, and win rate against the paired baseline at comparable budget points. History-level plots are therefore used to assess convergence behaviour and stability, whereas final best-score summaries are used to compare end-of-run outcomes.

All optimization metrics are also analysed conditionally on the initial-fitness buckets defined in Section 4.3. Bucket-conditioned summaries report the same best-score, improvement, paired-difference, and win-rate quantities within each starting-fitness range. This analysis distinguishes cases in which a method improves weak or medium starting genotypes from cases in which the starting genotype is already strong and therefore harder to surpass.

4.5 Experimental protocol

4.6 Reproducibility and computational setup

Chapter 5

Experimental Results

- 5.1 Autoencoder reconstruction and validity results
- 5.2 CoSO optimization results
- 5.3 Latent-space optimization results
- 5.4 Comparison with the Framsticks evolutionary baseline
- 5.5 Comparison between CoSO and Latent Space Optimization
- 5.6 Analysis by initial fitness ranges
- 5.7 Summary of results

Chapter 6

Discussion and Conclusions

- 6.1 Summary of experimental results
- 6.2 Strengths and limitations of Compatible Substitutions Optimization
- 6.3 Strengths and limitations of Latent Space Optimization
- 6.4 Comparative discussion
- 6.5 Threats to validity
- 6.6 Conclusions
- 6.7 Future work

Bibliography

- [1] Peter J. Bentley, Soo Ling Lim, Adam Gaier, and Linh Tran. Evolving through the looking glass: Learning improved search spaces with variational autoencoders. In *Parallel Problem Solving from Nature – PPSN XVII*, volume 13398 of *Lecture Notes in Computer Science*, pages 371–384. Springer, 2022.
- [2] Rafael Gómez-Bombarelli, Jennifer N. Wei, David Duvenaud, José Miguel Hernández-Lobato, Benjamín Sánchez-Lengeling, Dennis Sheberla, Jorge Aguilera-Iparraguirre, Timothy D. Hirzel, Ryan P. Adams, and Alán Aspuru-Guzik. Automatic chemical design using a data-driven continuous representation of molecules. *ACS Central Science*, 4(2):268–276, 2018.
- [3] Nikolaus Hansen. The CMA evolution strategy: A tutorial, 2016.
- [4] Geoffrey E. Hinton and Ruslan R. Salakhutdinov. Reducing the dimensionality of data with neural networks. *Science*, 313(5786):504–507, 2006.
- [5] Piotr Kaszuba, Maciej Komosiński, and Agnieszka Mensfelt. Automated development of latent representations for optimization of sequences using autoencoders. In *2021 IEEE Congress on Evolutionary Computation (CEC)*, pages 1360–1367, 2021.
- [6] Diederik P. Kingma and Max Welling. Auto-encoding variational bayes. In *Proceedings of the International Conference on Learning Representations*, 2014.
- [7] Matt J. Kusner, Brooks Paige, and José Miguel Hernández-Lobato. Grammar variational autoencoder. In *Proceedings of the 34th International Conference on Machine Learning*, volume 70 of *Proceedings of Machine Learning Research*, pages 1945–1954, 2017.
- [8] Paweł Liskowski, Krzysztof Krawiec, Nihat Engin Toklu, and Jerry Swan. Program synthesis as latent continuous optimization: Evolutionary search in neural embeddings. In *Proceedings of the 2020 Genetic and Evolutionary Computation Conference*, pages 359–367, 2020.
- [9] Danilo Jimenez Rezende, Shakir Mohamed, and Daan Wierstra. Stochastic backpropagation and approximate inference in deep generative models. In *Proceedings of the 31st International Conference on Machine Learning*, volume 32 of *Proceedings of Machine Learning Research*, pages 1278–1286, 2014.
- [10] Reuven Y. Rubinstein. The cross-entropy method for combinatorial and continuous optimization. *Methodology and Computing in Applied Probability*, 1(2):127–190, 1999.
- [11] David E. Rumelhart, Geoffrey E. Hinton, and Ronald J. Williams. Learning internal representations by error propagation. In David E. Rumelhart, James L. McClelland, and the PDP Research Group, editors, *Parallel Distributed Processing: Explorations in the Microstructure of Cognition, Volume 1: Foundations*, pages 318–362. MIT Press, Cambridge, MA, USA, 1986.

Appendix A

Supplementary Materials

- A.1 Additional experimental results
- A.2 Configuration details
- A.3 Examples of optimized genotypes



© 2026 Filip Rosiak

Poznań University of Technology
Faculty of Computing and Telecommunication
Institute of Computing Science

Typeset using L^AT_EX